

B AĞAÇLARI

B Ağacı Nedir:

Veritabanı sistemlerinde en çok kullanılan index türlerinden biridir. B Ağaçları birden fazla düğüm alabilen ve ikiden fazla çocuğu olabilen yapılardır, genellikle databaselerde dosya yapılarında kullanılır. B ve B+ ağaçları, diskteki dosyalarda hızlı arama yapılmasına olanak tanıyan diske uygun çok yollu ve dengeli ikili arama ağaçlarının bir çeşididir. B Ağaçları pointers ve linked list'lerin daha gelişmiş, diske uygun hale getirilmiş şeklidir, verileri her düğümün artan sırada anahtarlar içermesini sağlayacak şekilde organize eder. B+ Ağaçları buna benzer olsa da, daha verimli bir yaklaşım olarak verileri yalnızca yaprak düğümlerde depolar.

Veri Tabanlarında Kullanımı:

Sıralı Dosya Arama Süreci: Sıralama ve arama algoritmaları, sıralama gösterimi kullanılarak gerçekleştirilmesi gereken karşılaştırma işlemlerinin sayısı ile karakterize edilebilir.

Endeks Performansı: B ağacı indeksi, performansı iyileştirmek için kullanılabilir. B-ağacı indeksi, bir veritabanını sabit boyutlu bloklara veya sayfalara bölen çok seviyeli bir ağaç yapısı oluşturur. Bu ağacın her seviyesi, adres konumu aracılığıyla bu sayfaları birbirine bağlamak için kullanılabilir; bu da bir sayfanın (düğüm veya iç sayfa olarak bilinir) en alt seviyedeki yaprak sayfalarla başka bir sayfaya referans vermesine olanak tanır.

Eklemler ve Silmeler:Veritabanı değışmiyorsa, indeksi derlemek basittir ve indeksin asla değıştirilmesine gerek kalmaz. Değışiklikler varsa, veritabanını ve indeksini yönetmek ek hesaplama gerektirir . Veritabanından kayıtları silmek nispeten kolaydır. İndeks aynı kalabilir ve kayıt sadece silinmiş olarak işaretlenebilir. Veritabanı sıralı halde kalır. Çok sayıda tembel silme işlemi varsa , arama ve depolama daha az verimli hale gelir. Bloкта yer olduğu sürece hem eklemeler hem de silmeler hızlıdır.

B-ağacı yukarıda açıklanan tüm fikirleri kullanır. Özellikle, bir B-ağacı:

- Ardışık gezinme için anahtarları sıralı halde tutar.
- Disk okuma sayısını en aza indirmek için hiyerarşik bir indeks kullanır.
- Kısmen dolu blokları kullanarak ekleme ve silme işlemlerini hızlandırır.
- İndeks dengesini özyinelemeli bir algoritma ile korur.

Ayrıca, bir B-ağacı, iç düğümlerin en az yarı dolu olmasını sağlayarak israfı en aza indirir. Bir B-ağacı, keyfi sayıda ekleme ve silme işlemini işleyebilir.

Dosya Sistemlerinde Kullanımı:

B-tree (veya varyantları), veritabanlarındaki kullanımına ek olarak, belirli bir dosyadaki herhangi bir bloğa hızlı ve rastgele erişim sağlamak amacıyla dosya sistemlerinde de kullanılır. Buradaki temel sorun, *i* dosya bloğu adresini bir disk bloğu adresine dönüştürmektir. Bazı eski ve yüksek düzeyde özelleşmiş işletim sistemleri, dosya ilk oluşturulduğunda uygulamanın o dosyanın maksimum boyutunu tahsis etmesini gerektiriyordu. Böylece dosya, ardışık disk blokları olarak ayrılabilirdi. Bu durumda, *i* dosya bloğu adresini bir disk bloğu adresine dönüştürmek için, işletim sistemi basitçe *i* dosya bloğu adresini dosyayı oluşturan ilk disk bloğunun adresine ekler. Bu yöntem basittir, ancak dosya oluşturulduğu boyutu aşamaz. Tüm modern ve yaygın işletim sistemleri ise bir dosyanın büyümesine izin verir. Bunun sonucunda oluşan disk blokları ardışık olmayabilir, bu nedenle mantıksal blokları fiziksel bloklara eşlemek daha karmaşık bir işlemdir.

Günümüzde kullanılma nedeni RAM'den veri okumak nanosaniyeler, sabit diskten veri okumak ise milisaniyeler sürer. Diskten veri okunurken bayt bayt değil, "bloklar" halinde okunur. Tek bir disk okuma işlemiyle binlerce anahtar belleğe çekilir ve diske erişim sayısı minimuma indirilir.

Kütüphanede katları ve bölümleri gösteren yönlendirme tabelaları var.

B ağacı:

Yönlendirme tabelalarının yanına kitapları dizmişler. Örneğin tabelada *"A-C arası 2. kata"* yazıyor, ama hemen o tabelanın altına aranan kişinin kitabını fiziksel olarak yerleştirmişler. Eğer şanslıysanız ve aradığınız kitap kişinin kitabıysa, hiç üst raflara ilerlemeden kitabı tam o tabelanın önünde bulup alırsınız, hızlıdır. Gerçek kitaplar ve dosyalar çok kalın ve ağır olduğu için, tabelanın etrafındaki boşluk hemen dolar. Bu yüzden giriş katına koyabileceğiniz yönlendirme tabelası sayısı çok azalır. Kütüphane büyüdükçe sistem tıkanır; yönlendirme yapmak yerine alanı kitap depolamak için harcamış olursunuz.

B+ ağacı:

Şimdi aynı kütüphanenin daha modern ve optimize edilmiş halini düşünelim. Bu kütüphanenin altın kuralı çok nettir: Üst katlarda ve giriş holünde kesinlikle fiziksel kitap bulunamaz. Giriş ve ara katlarda sadece incecik kağıtlardan oluşan devasa yönlendirme panoları vardır. *"Bilgisayar Bilimleri bodrum kata, Veri Yapıları 1. kata"* gibi sadece yol gösteren bilgiler bulunur. Hiç kalın kitap olmadığı için, tek bir panoya binlerce yönlendirme sığdırılabilir. Aradığınız kitabın tam olarak hangi odada olduğunu bulmak tek bir bakışla saniyeler sürer. Tüm fiziksel kitaplar ve öğrenci dosyaları sadece ve sadece en alt kattaki (yaprak düğümler) devasa depodadır.

Bu veri yapıları bize hızla birlikte daha az maliyetli ve dengeli bir yapı sunmuştur. Ölçeklenebilir olduğundan da diski verimli kullanarak düzgün okuma, yazma ve arama yapılmasını sağlar. Bir sunucuda milyonlarca veri yönetilirken en verimli yapı B ağaçları yapısıdır. Düğüm boyutundaki esneklik de avantajları arasındadır.

Büyük veri çağında milyonlarca veriyi yönetirken doğru yapıyı seçmemek sistemin çökmesine ya da işlemlerin dakikalar almasına yol açar. Belirli uygulamalarda veri hacmi çok hızlı şekilde artıyor, bu artmada veri setlerinde aranan veriler aralıklarda belirtilerek kolay şekilde bulunabiliyor.

Gelecekte bu yapının üzerine eklenerek kullanılmaya devam edileceğini düşünüyorum. Başta B ağacının, sonrasında da B+ ağacını çıkması gibi. Bellek sıralamasında Cache'den iyisi çıkana dek bu yapının kullanılacağını düşünüyorum.